HANDS-ON GUIDE · APPS/BPMN-ENGINE

Modeling processes with Mini BPMN

How to read, draw, wire, deploy and run your own process flows with the modeler, engine and cockpit in this repository.



the loop you repeat for every process; each new deploy becomes a new version

1 The mental model
2 Getting around the tool
3 The building blocks
4 Tutorial: build a process from scratch
5 Variables and conditions
6 Pattern cookbook
7 Deploying, versioning and operating
8 Troubleshooting
9 Checklist and quick reference

Everything here matches the code in `apps/bpmn-engine`: the parser (`src/engine/bpmn-parser.ts`), the engine (`src/engine/bpmn-engine.service.ts`) and the browser UI (`public/index.html`). The tutorial process in section 4 was validated against the real parser.

1 The mental model

A process in this tool has two halves, and you always write both:

	THE DIAGRAM (`.BPMN`)	THE HANDLERS (TYPESCRIPT)
Answers	**What** happens and **in which order**: steps, decisions, parallel work, waiting, error paths, undo.	**How** one step is done: call a service, compute a value, send an email.
Lives in	The modeler, or `apps/bpmn-engine/processes/*.bpmn`	`apps/bpmn-engine/src/handlers/*.ts`
Linked by	A name. The service task says `mini:handler="inspection.assess"`; the code calls `registry.register('inspection.assess', fn)`.	
Changing it	Redraw and deploy. You get a new version, no restart.	Edit code. `npm run start:bpmn` runs in watch mode and restarts.

Tokens: how the engine reads your drawing

When you start an instance, the engine puts a **token** on the start event. It then repeatedly takes a token that is ready and does what the token's element says: run a handler, evaluate a condition, split into several tokens, or **park** and wait. After every step the instance is saved to SQLite. When no tokens are left, the instance is **COMPLETED**.

A token stops moving for one of four reasons, and the cockpit colours each one:

TOKEN STATE	WHY IT STOPPED	WHAT MOVES IT AGAIN
WAITING (amber)	User task, timer, or a parallel join waiting for its other branches	A person completes the form · the timer fires · the missing branch arrives
INCIDENT (red)	A technical failure: handler crashed, service down, a condition threw, no flow matched	An operator fixes the cause (and maybe the variables), then clicks **Retry**
EXECUTING (blue)	A service task handler is running right now	The handler returning (or the engine re-running it after a crash)
gone	It reached an end event	—

> **Think in tokens while you draw**
> For every element ask: “when a token arrives here, what happens, and where does it go next?” If the answer is “nowhere”, you need an end event. If two tokens can arrive, ask whether they should wait for each other (parallel join) or not (exclusive merge).

The data: process variables

Every instance carries one JSON object, the **variables**. You pass the first ones when you start the instance; every handler gets a copy and returns the keys it wants to add or replace; user task forms add their fields; conditions on flows read them. Section 5 covers the details.

2 Getting around the tool

Start it

```
npm run start:participants      # terminal 1: the 5 services the shipment saga calls (optional
for your own processes)
npm run start:bpmn              # terminal 2: the BPMN tool, in watch mode
open http://localhost:3200/     # port: BPMN_PORT, default 3200
npm run db:reset                # wipe all deployments and instances (stop the engine first)
```

On boot the engine deploys every file in `apps/bpmn-engine/processes/`. The page has a sidebar and two tabs:

AREA	WHAT YOU DO THERE
Deployed processes (sidebar)	Each process key with its latest version. *Open in modeler* loads it for editing; *Select to start* picks it for the panel below.
Start an instance	Edit the starting variables as JSON (presets exist for the two samples) and click **Start**.
Instances	The last 100 instances with status, incident count and where they wait. Click one to open it in the cockpit.
Modeler tab	The canvas (bpmn-js, the library inside Camunda Modeler), a toolbar and a properties panel on the right.
Cockpit tab	The diagram of the instance's exact version with live tokens, *Open work* (forms, incidents, timers), variables and the history.

Modeler toolbar

New diagram	An empty process with key `my-process` and one start event. Rename the key first (see below).
Show XML	Switch to the raw XML. You may edit it; *Back to diagram* imports your edits. Use it for things the panel can't set (error codes).
Download .bpmn	Save the diagram as `<key>.bpmn`. Put it in `processes/` to keep it in git.
Deploy	Send it to the engine. Either “Deployed … as version N” or a list of everything wrong with it.

Drawing with bpmn-js

Palette (left edge)	Drag an element onto the canvas: events (circles), task (rounded box), gateway (diamond).
Context pad	Click any element: small icons appear next to it. Use them to **append** the next element (it is connected for you), draw a **connection** (arrow icon), change the type with the **wrench**, or delete it.
Wrench (replace)	How you turn a plain task into a *Service Task* or *User Task*, a diamond into a *Parallel Gateway*, a circle into a *Timer* event.

Boundary events	Drop an intermediate event *onto the border* of a task (the border highlights), then use the wrench to make it an error or compensation boundary event.
Labels	Double-click to type a name, or use *Name* in the properties panel.
Navigation	`Ctrl`+scroll to zoom, drag empty space to pan, `Ctrl`+`Z` to undo, `Del` to delete, drag arrow bend points to tidy up.

The properties panel

It shows the fields this engine needs for the selected element. With **nothing selected** it shows the process itself: **Process key (id)** and **Name**. The key is how you start instances and how versions are grouped; set it before your first deploy.

SELECTED	FIELDS
Service task	**mini:handler**, with suggestions from `GET /api/handlers` (names registered when the page loaded)
User task	**mini:formFields**, e.g. `approved:boolean,comment:string,amount:number`
Sequence flow	**Condition** such as `${quote.amount > 10000}`; if it leaves an XOR gateway, the **default flow** checkbox
Timer event	**Timer duration**, ISO-8601: `PT30S`, `PT5M`, `PT1H30M`, `P2D`
Error boundary	Shows which error code it catches (set it in XML, see 6.5)

3 The building blocks

This engine runs a deliberate subset of BPMN 2.0. Anything else (subprocess, call activity, script/send/receive task, inclusive or event-based gateway, message or signal events) is **refused at deploy time** with a clear message, never silently ignored.

	ELEMENT AND WHAT IT DOES HERE	HOW TO MAKE IT IN THE MODELER	XML
	**Start event** Where every instance begins. Exactly one per process; no incoming flows.	Palette: thin circle. Nothing to configure.	`<bpmn:startEvent>`
	**End event** Consumes the token. When no tokens are left, the instance is **COMPLETED**. Use as many as you like.	Palette: thick circle.	`<bpmn:endEvent>`
	**Terminate end event** Ends the *whole* instance at once, removing tokens still waiting on other branches.	End event → wrench → *Terminate End Event*.	`terminateEventDefinition`
	**Task (plain)** Does nothing and moves on. Handy as a placeholder while sketching.	Palette: rounded box.	`<bpmn:task>`
	**Service task** Runs your TypeScript handler; its return value is merged into the variables.	Wrench → *Service Task*. Set **mini:handler** (dropdown lists registered handlers).	`mini:handler="x.y"`
	**User task** **Wait state.** A form appears in the cockpit; submitted fields become variables.	Wrench → *User Task*. Set **mini:formFields**.	`mini:formFields="ok:boolean"`
	**Exclusive gateway (XOR)** Split: takes the **first** outgoing flow whose condition is true, else the default. Merge: just passes through.	Palette: diamond. Put conditions on the outgoing *flows*, tick **default flow** on one.	`default="Flow_id"`
	**Parallel gateway (AND)** Split: one token per outgoing flow. Join: waits until **every** incoming flow has delivered a token.	Diamond → wrench → *Parallel Gateway*.	`<bpmn:parallelGateway>`
	**Timer catch event** **Wait state.** Parks the token for an ISO-8601 duration. Survives engine restarts.	Intermediate event → wrench → *Timer Intermediate Catch Event*. Set **Timer duration**.	`<timeDuration>PT30S`
	**Error boundary event** Sits on a task's border. If the handler throws `BpmnError`, the task is abandoned and the token leaves from here.	Drag an intermediate event onto the task border → wrench → *Error Boundary Event*.	`errorEventDefinition`

	**Compensation boundary** Links a task to the task that undoes it (dotted association). Never on the normal path.	On the border → wrench → *Compensation Boundary Event*; connect it to the undo task.	`compensateEventDefinition`
	**Compensation throw event** Runs the undo tasks of all **completed** activities, newest first, then continues.	Intermediate event → wrench → *Compensation Intermediate Throw Event*.	`intermediateThrowEvent`

Connections

Sequence flow (solid arrow)	“After this, go there.” The only thing tokens travel along.
Conditional flow (small diamond at the start)	A flow out of an XOR gateway with a `${…}` condition.
Default flow (slash at the start)	The XOR gateway's fallback when no condition is true. At most one per gateway.
Association (dotted)	Links a compensation boundary event to its undo task. Tokens never travel along it.

> **Two implicit behaviours to know**
> **Several arrows out of a task** act like a parallel split: every flow gets a token, and conditions on them are ignored. **Several arrows into a task** act like an exclusive merge: the task runs once per arriving token. Prefer explicit gateways; the diagram then says what you mean.

4 Tutorial: build a process from scratch

We will build **Customs inspection**: when a container arrives, assess its risk. High-risk containers are inspected by a person; if the inspection fails, the container is put on hold. Otherwise it is released, and in parallel the free storage period starts counting down. It touches almost every element: service and user tasks, both gateway types, a timer, conditions and default flows.

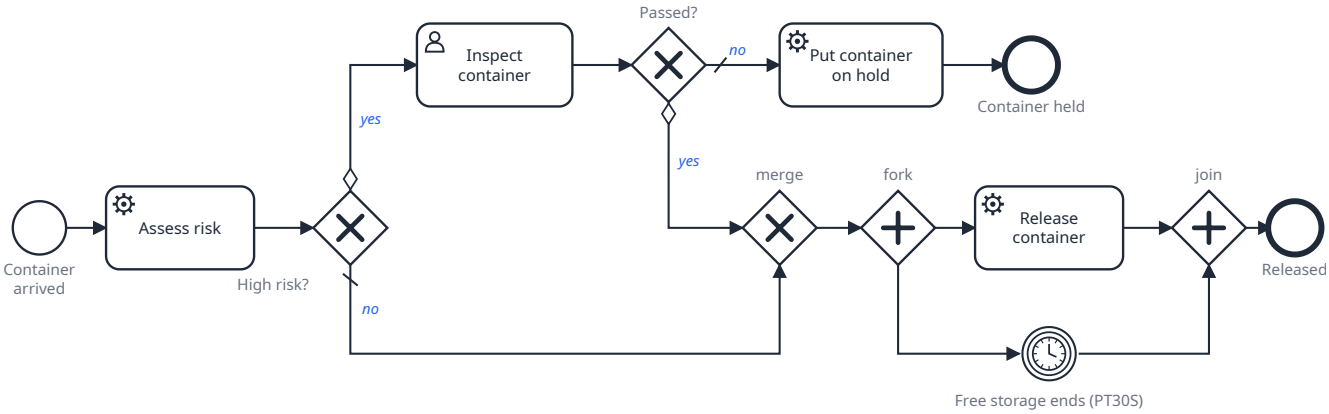


Figure 1: the finished process. Gear = service task, person = user task, small diamond = condition, slash = default flow.

4.1 Design it on paper first

Before touching the modeler, write down the steps, who or what does each one, and the data each decision needs:

STEP	KIND	NEEDS	PRODUCES
Assess risk	service: `inspection.assess`	`shipment`	`risk.score`
High risk?	XOR	`risk.score`	—
Inspect container	user task	a person	`passed`, `notes`
Passed?	XOR	`passed`	—
Put on hold / Release	service: `inspection.hold` / `.release`	`notes`	`heldAt` / `releasedAt`
Free storage ends	timer, PT30S (days in real life)	—	—

The “Needs” column is what makes conditions safe: every variable a condition reads must exist by the time the token gets there.

4.2 Write the handlers

Create `apps/bpmn-engine/src/handlers/inspection-handlers.ts`. A handler gets `{ instanceId, nodeId, variables }` (a copy) and returns an object that is merged into the variables:

```
import { Injectable, Logger, OnModuleInit } from '@nestjs/common';
import { HandlerRegistry } from '../engine/handlers';

/** Glue code for processes/customs-inspection.bpmn. */
@Injectable()
export class InspectionHandlers implements OnModuleInit {
  private readonly log = new Logger('inspection');

  constructor(private readonly registry: HandlerRegistry) {}

  onModuleInit() {
    // Read variables, return the ones you want to add or change.
    this.registry.register('inspection.assess', ({ variables }) => {
      const s = variables.shipment ?? {};
      const score = String(s.hsCode ?? '').startsWith('93') ? 90   // weapons: always inspect
                  : (s.cargoValueEur ?? 0) > 100_000       ? 75   // high value
                  : 20;
      return { risk: { score } };
    });

    this.registry.register('inspection.hold', ({ instanceId, variables }) => {
      this.log.log(`[${instanceId.slice(0, 8)}] container held: ${variables.notes ?? '-'}`);
      return { heldAt: new Date().toISOString() };
    });

    this.registry.register('inspection.release', () => ({ releasedAt: new Date().toISOString()
}));
  }
}
```

Register the class as a provider so Nest instantiates it:

```
// apps/bpmn-engine/src/app.module.ts
import { InspectionHandlers } from './handlers/inspection-handlers';
...
  providers: [BpmnEngine, HandlerRegistry, FreightHandlers, InspectionHandlers,
SampleProcesses],
```

The engine restarts on save. **Reload the browser page** so the handler dropdown in the properties panel picks up the new names.

4.3 Draw it

1. **New diagram.** Click empty canvas and set *Process key* to `customs-inspection`, *Name* to `Customs inspection`.
2. Click the start event, name it *Container arrived*. From its context pad, **append a task**. Wrench → *Service Task*. Name *Assess risk*, handler `inspection.assess`.
3. From the task, append a **gateway** (exclusive is the default). Name it *High risk?*
4. From the gateway, append a task above it: wrench → *User Task*, name *Inspect container*, form fields `passed:boolean,notes:string`.
5. Select the arrow from *High risk?* to *Inspect container*. Condition `${risk.score > 70}`, name it *yes*.
6. After the user task, append another XOR gateway *Passed?*. Append a service task *Put container on hold* (`inspection.hold`) and after it an **end event** *Container held*. Select that *Passed?* → *hold* arrow and tick **default flow**.

7. Drop a new exclusive gateway to the right of *High risk?* (the **merge**). Connect *High risk?* to it, select that arrow, tick **default flow**, name it *no*. Connect *Passed?* to the merge too, with condition `${passed === true}`.
8. From the merge, append a gateway and wrench it into a **Parallel Gateway** (the fork). Append service task *Release container* (`inspection.release`).
9. From the fork, also append an **intermediate event**; wrench → *Timer Intermediate Catch Event*; duration `PT30S` .
10. Add a second parallel gateway (the join). Connect *Release container* and the timer to it. Append an end event *Released*.

> **Why the separate XOR merge in step 7?**
> The low-risk path and the "passed" path are *alternatives*: only one token ever arrives. If you wired both straight into the parallel fork, that gateway would have two incoming flows and so act as a **join**, waiting forever for the branch that never comes. Rule: **merge alternative paths with an XOR; join only what an AND forked.**

4.4 Check the XML

Click **Show XML**. Ignoring the `bpmndi` section (just x/y coordinates for drawing; the engine never reads it), the process should look like this. Note that `>` is escaped as `>` inside XML; the modeler does that for you, but remember it when editing by hand.

```
<bpmn:process id="customs-inspection" name="Customs inspection" isExecutable="true">
  <bpmn:startEvent id="Start_Arrived" name="Container arrived" />
  <bpmn:serviceTask id="Task_Assess" name="Assess risk" mini:handler="inspection.assess" />
  <bpmn:exclusiveGateway id="Gw_Risk" name="High risk?" default="Flow_LowRisk" />
  <bpmn:userTask id="Task_Inspect" name="Inspect container"
                 mini:formFields="passed:boolean,notes:string" />
  <bpmn:exclusiveGateway id="Gw_Passed" name="Passed?" default="Flow_Failed" />
  <bpmn:serviceTask id="Task_Hold" name="Put container on hold" mini:handler="inspection.hold"
/>
  <bpmn:endEvent id="End_Held" name="Container held" />
  <bpmn:exclusiveGateway id="Gw_Merge" />
  <bpmn:parallelGateway id="Gw_Fork" />
  <bpmn:serviceTask id="Task_Release" name="Release container"
mini:handler="inspection.release" />
  <bpmn:intermediateCatchEvent id="Timer_Free" name="Free storage ends">
    <bpmn:timerEventDefinition><bpmn:timeDuration>PT30S</bpmn:timeDuration>
</bpmn:timerEventDefinition>
  </bpmn:intermediateCatchEvent>
  <bpmn:parallelGateway id="Gw_Join" />
  <bpmn:endEvent id="End_Released" name="Released" />

  <bpmn:sequenceFlow id="Flow_HighRisk" name="yes" sourceRef="Gw_Risk"
targetRef="Task_Inspect">
    <bpmn:conditionExpression>${risk.score &gt; 70}</bpmn:conditionExpression>
  </bpmn:sequenceFlow>
  <bpmn:sequenceFlow id="Flow_LowRisk" name="no" sourceRef="Gw_Risk" targetRef="Gw_Merge" />
  <bpmn:sequenceFlow id="Flow_Passed" name="yes" sourceRef="Gw_Passed" targetRef="Gw_Merge">
    <bpmn:conditionExpression>${passed === true}</bpmn:conditionExpression>
  </bpmn:sequenceFlow>
  <bpmn:sequenceFlow id="Flow_Failed" name="no" sourceRef="Gw_Passed" targetRef="Task_Hold" />
  <!-- ...plus the plain flows between the other elements -->
</bpmn:process>
```

4.5 Deploy

Click **Deploy**. You should see “Deployed customs-inspection as version 1”. If the engine refuses, it lists every problem at once, for example:

```
The engine refused the diagram:
 • exclusiveGateway "High risk?": flows Flow_0x1 need a condition or must be the default flow
 • serviceTask "Put container on hold" is a dead end (no outgoing flow); use an end event
 • endEvent "Container held" is unreachable (no incoming flow)
```

Fix them on the canvas and deploy again. Section 8 lists every message. Then **Download .bpmn** and save it as `apps/bpmn-engine/processes/customs-inspection.bpmn` so it is in git and redeployed on every boot.

4.6 Run it: three test cases

Select to start on the new process, paste the variables, click **Start**. The cockpit opens on the instance.

START VARIABLES	WHAT YOU SHOULD SEE
`{"shipment": {"hsCode": "854143"}}`	Score 20. *no* path, fork: *Release* finishes, the join shows “1/2 branches arrived” until the timer fires after 30 s, then **COMPLETED** at *Released*.
`{"shipment": {"hsCode": "930100"}}`	Score 90. Token waits (amber) on *Inspect container*; a form appears under **Open work**. Tick `passed`, complete: same ending as above.
Same, but leave `passed` unticked	Default flow to *Put container on hold*, ends at *Container held*. The engine log prints your notes.
`{"shipment": {"cargoValueEur": 250000}}`	Score 75, also inspected: proof the second rule works.

Read the **History** panel for each run: it records every node completed, every flow taken and every wait, so it is the quickest way to understand *why* the token went where it did.

5 Variables and conditions

Where variables come from

SOURCE	HOW IT IS MERGED
Start of the instance	The JSON you pass becomes the initial variables.
Service task handler	Returned object is merged with `Object.assign`: **top-level keys are replaced**, not deep-merged. Return `void` to change nothing.
User task form	Each form field becomes a top-level variable. `boolean` = checkbox (unticked sends `false`), `number` = number input, `string` = text.
Caught business error	The engine sets `lastError = { code, message, nodeId }` before following the error boundary.
Operator in the cockpit	**Edit** on the variables panel **replaces all** variables with the JSON you save.

> **Shallow merge bites**
> If `context = { ocean: … }` and a handler returns `{ context: { customs: … } }`, the ocean entry is gone. Spread the old value yourself, as `freight-handlers.ts` does: `return { context: { ...variables.context, customs: result } }`.

Writing conditions

A condition is `${ … }` around a **JavaScript expression**. Every top-level variable is in scope by name.

```
${quote.amount > 10000}                    nested properties
${approved}                                a boolean from a user task form
${passed === true && notes.length > 0}     combine with && || !
${shipment.containerType === '40HC'}       strings in single quotes
${typeof discount !== 'undefined'}         test whether a variable exists at all
${(order?.items ?? []).length > 3}         optional chaining for things that may be missing
```

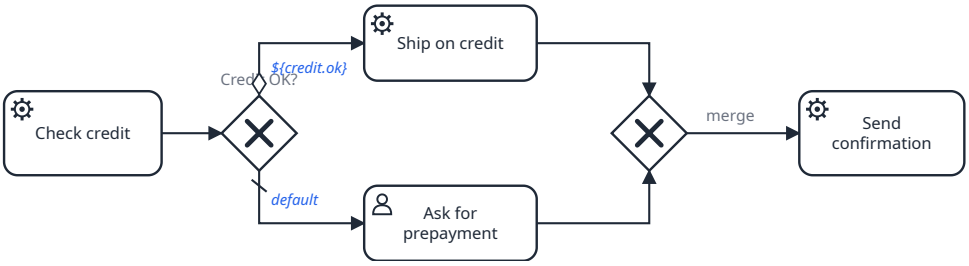
- **Order matters.** The gateway checks its outgoing flows in the order they appear in the XML and takes the **first** true one. Make conditions mutually exclusive so order doesn't matter.
- **Always give a default flow** unless the conditions provably cover every case. No true condition and no default = incident.
- **A missing top-level variable throws** (`ReferenceError: passed is not defined`), which becomes an incident "condition failed". Fix by editing variables in the cockpit and retrying, or guard with `typeof`.
- Conditions only matter on flows leaving an **exclusive gateway**. Anywhere else they are ignored.

> **Keep logic in handlers, keep decisions readable**
> If a condition gets long, compute the answer in the preceding service task (`return { needsInspection: … }`) and test `${needsInspection}`. The diagram stays readable and the rule is unit-testable TypeScript.
>
> *Security note:* conditions run as real JavaScript on the server. That is fine for learning, but anyone who can deploy can run code.

6 Pattern cookbook

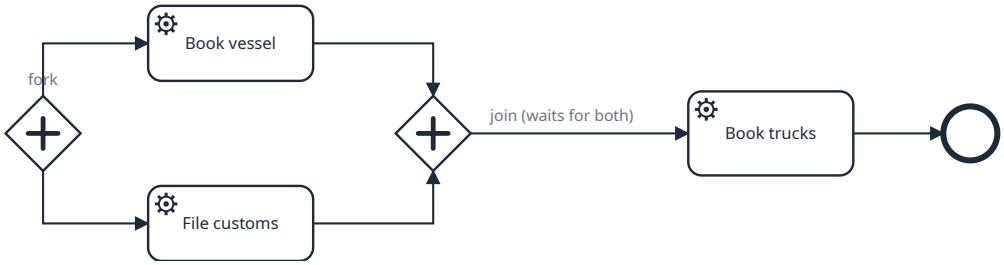
Most real processes are combinations of the patterns below. Each shows the shape to draw and the rule that makes it work.

6.1 Decision and merge (XOR)



One path out of several. Conditions on the outgoing flows, one default. When the paths come back together, use another XOR (or connect straight into the next task).

6.2 Parallel work (AND fork / join)



Independent steps that may run side by side. The join waits until a token arrived on **every** incoming flow, then continues with one token. Every branch from the fork must reach the join, or the join waits forever. (The engine runs branches one after the other, but in BPMN terms they are concurrent.)

6.3 Human step and approval

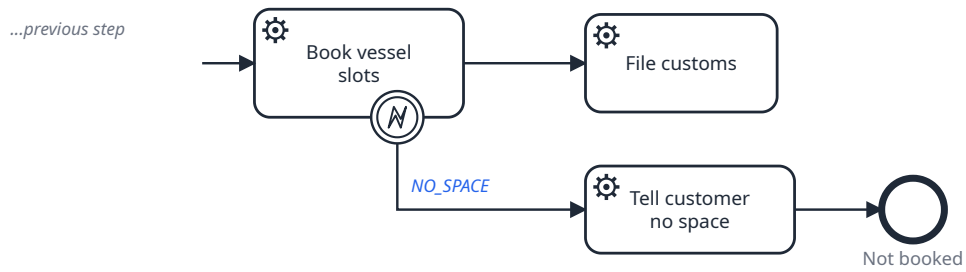
A user task with form fields, followed by an XOR testing the answer. See *Manager approves quote → Approved?* in `processes/quote-approval.bpmn`: fields `approved:boolean,comment:string`, condition `${approved}`, rejection as the default. The instance can wait for days: it is in SQLite, not memory.

6.4 Waiting (timer)



A timer catch event parks the token for a duration (`PnDTnHnMnS`). A background job checks due timers every second, and timers survive restarts. Only *durations* are supported, not dates or cycles. Timer *boundary* events (“escalate if not done in 2 days”) are not supported yet.

6.5 Business errors (error boundary)



Separate the two kinds of failure: the **diagram** handles expected business outcomes; the **operator** handles technical ones.

IN THE HANDLER	ENGINE DOES	EXAMPLE
`throw new BpmnError('CODE', msg)`	Follows a matching error boundary on that task; no match = incident	vessel full, customs rejects, credit limit
`throw new Error(msg)` (anything else)	Incident: the token stops until someone clicks Retry	service down, timeout, bug

```
import { BpmnError } from '../engine/handlers';

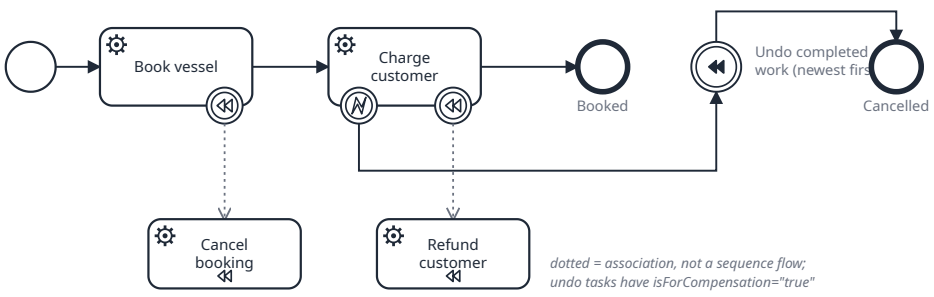
this.registry.register('ocean.book', async ({ variables }) => {
  const reply = await carrier.book(variables.shipment); // network down → Error → incident
  if (!reply.ok) throw new BpmnError('NO_SPACE', reply.reason); // a business "no"
  return { booking: reply };
});
```

An error boundary with no error reference **catches every code**. To catch one specific code, add it in **Show XML**:

```
<!-- top level, next to <bpmn:process> -->
<bpmn:error id="Error_NoSpace" name="No space" errorCode="NO_SPACE" />

<!-- inside the process -->
<bpmn:boundaryEvent id="Err_Ocean" attachedToRef="Task_Ocean">
  <bpmn:errorEventDefinition errorRef="Error_NoSpace" />
</bpmn:boundaryEvent>
```

6.6 Undo on failure (compensation, BPMN's saga)

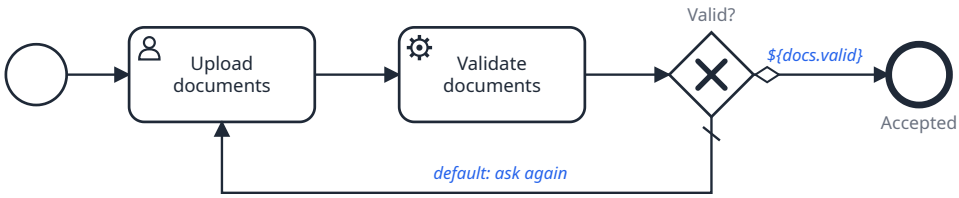


1. For each step that changes something, put a **compensation boundary event** on it.

2. Draw the undo task (a service task, e.g. `ocean.cancel`) below it and connect the boundary event to it. The connection becomes a dotted **association**. In **Show XML**, check that the undo task has `isForCompensation="true"` and no sequence flows.
3. Put **error boundaries** on the steps that can fail; lead them to one **compensation throw event**, then an end event.

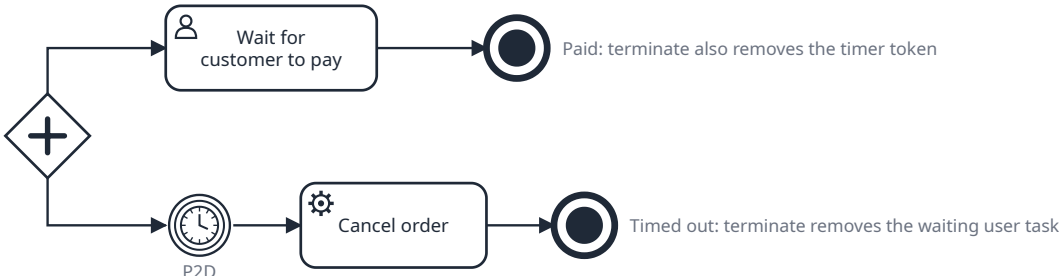
The throw event undoes only **completed** steps, newest first. The step that failed is not undone (it never completed). If an undo handler throws, you get an incident; on retry, steps already undone are skipped. Undo handlers must be idempotent. `processes/shipment-saga.bpmn` is the complete worked example.

6.7 Loop until valid



A flow may point backwards. Make sure every loop passes a wait state (user task, timer): the engine raises an incident after 1,000 steps without one, assuming an endless loop.

6.8 Stop everything (terminate end)



A normal end event only consumes its own token. A **terminate** end event removes every other token too. Here it is a race: whichever branch finishes first ends the instance. With plain end events, paying would leave the timer running and the order would still be cancelled two days later.

7 Deploying, versioning and operating

Three ways to deploy

Deploy button	Fastest while iterating.
Files in `processes/`	Every `.bpmn` there is deployed when the engine boots. Recommended for anything you want to keep: it is reviewed and versioned in git.
REST API	For scripts and CI. Same endpoints the UI uses:

```
# deploy (same as the Deploy button)
curl -X POST localhost:3200/api/processes -H 'content-type: application/json' \
     -d "$(jq -Rs '{xml: .}' apps/bpmn-engine/processes/customs-inspection.bpmn)"

# start the latest version with some variables
curl -X POST localhost:3200/api/processes/customs-inspection/start \
     -H 'content-type: application/json' -d '{"variables": {"shipment": {"hsCode": "930100"}}}'

# what is deployed / which handlers exist
curl localhost:3200/api/processes
curl localhost:3200/api/handlers
```

How versions work

- A process is identified by its **key** (the `id` of `<bpmn:process>`). Deploying a key again makes **version N+1**.
- Deploying **byte-identical** XML does not create a new version, so rebooting does not pile up versions.
- **Start** always uses the latest version. **Running instances stay on the version they started with**, and the cockpit draws that version.
- So you can change a process while instances are waiting in it. But if you rename a handler, old instances still call the old name: keep it registered until they finish.
- Changing the key creates a new, unrelated process; changing element ids is fine.

Operating instances in the cockpit

SITUATION	WHAT TO DO
Form under **Open work**	Fill it in and **Complete task**. Fields become variables and the token moves on.
Incident card	Read the message. Fix the cause: start the service, fix a handler, or **Edit** the variables. Then **Retry**: the element runs again.
Timer card	Shows a countdown. Nothing to do.
Stuck for good	**Cancel instance** removes all tokens. Nothing is compensated, so clean up by hand if needed.

> **Handlers run at least once**
> The engine saves the token as EXECUTING before calling a handler. If it crashes mid-call, the handler runs **again** after restart; a Retry after a timeout also runs it again. Make handlers idempotent: use the instance id as the idempotency key, as the freight handlers do with `sagaId`.

8 Troubleshooting

Deploy-time messages

MESSAGE	FIX
`expected exactly one start event, found N`	Exactly one start event per process.
`… is unreachable (no incoming flow)`	Connect something to it, or delete it.
`… is a dead end (no outgoing flow); use an end event`	Every path must finish at an end event.
`serviceTask … needs a mini:handler attribute`	Set *mini:handler*. A plain task needs none.
`exclusiveGateway …: flows … need a condition or must be the default flow`	Give each outgoing flow a condition, or tick *default flow* on one.
`… needs a <timeDuration> (e.g. PT10S)`	Set the timer duration.
`only timer catch events are supported`	Message and signal catch events aren't implemented. Use a user task or timer.
`only error and compensation boundary events are supported`	Timer or message boundaries aren't implemented.
`subProcess "…" is not supported by this mini engine`	Flatten it into the main process. Same for call activity, script/send/receive/manual/business-rule task, inclusive/event-based/complex gateways.
`sequence flow … connects unknown elements`	Usually a hand-edited XML typo in `sourceRef` / `targetRef`.
`… must not have incoming/outgoing flows`	Nothing flows into a start event or out of an end event.

Runtime incidents

INCIDENT	CAUSE AND FIX
`no handler registered for "x.y"`	Typo in *mini:handler*, or the provider isn't in `app.module.ts`. Register it, then Retry.
`condition failed: … is not defined`	A condition reads a variable that doesn't exist yet. Edit variables and Retry; fix the design (section 5).
`no condition of "…" is true and there is no default flow`	Add a default flow. For this instance: edit variables so a condition matches, then Retry.
`uncaught BPMN error CODE`	The handler threw `BpmnError` but the task has no matching error boundary.
`… is not running (ECONNREFUSED)` / `did not reply`	A participant service is down or slow. Start it, then Retry.
`more than 1000 steps without a wait state`	Endless loop. Put a user task or timer inside the loop, or fix the exit condition.

Instance never finishes, no incident

- **Join shows “1/2 branches arrived”** and nothing else moves: a parallel gateway with several incoming flows is waiting for a branch that took another path. Use an XOR merge (section 4.3).
- **Waiting on a user task**: someone has to complete it under **Open work**.

9 Checklist and quick reference

Before you deploy

- ☐ Process key set (not `my-process`), and a readable name.
- ☐ One start event; every path ends in an end event.
- ☐ Every service task has a *mini:handler* that is registered in code.
- ☐ Every XOR split has conditions plus one default flow; conditions don't overlap.
- ☐ Every variable a condition reads is set before the token gets there.
- ☐ Every AND join closes an AND fork, with the same number of branches. Alternatives are merged with an XOR.
- ☐ Expected business failures are `BpmnError`s with an error boundary; technical ones are left as incidents.
- ☐ Every step that changes the outside world is idempotent, and has an undo task if the process can fail after it.
- ☐ Loops pass through a wait state.
- ☐ The diagram is downloaded into `processes/` and committed.

Cheat sheet

I WANT TO…	DRAW / WRITE
Call code	Service task, `mini:handler="area.action"` + `registry.register('area.action', fn)`
Ask a person	User task, `mini:formFields="name:type,…"` (string | number | boolean)
Choose one path	XOR + `${expression}` on flows + one default flow
Do things side by side	AND fork … AND join
Wait	Timer catch event, `PT30S` / `PT5M` / `P1D`
Handle a business “no”	`throw new BpmnError('CODE')` + error boundary event
Undo what was done	Compensation boundary + association to an undo task + compensation throw event
Stop all branches	Terminate end event
Change a running design	Edit + Deploy: new version; old instances keep theirs

Where to look in the code

`src/engine/bpmn-parser.ts`	What is accepted and every deploy-time check
`src/engine/bpmn-engine.service.ts`	`execute()`: what each element does to a token
`src/engine/expressions.ts`	Condition evaluation and timer durations
`src/handlers/freight-handlers.ts`	Handler examples, BpmnError vs incident
`processes/*.bpmn`	*quote-approval*: XOR, AND, user tasks, timer · *shipment-saga*: errors, compensation